

- TomatoMall Redis 功能说明文档
 - 一、概述
 - 二、Redis 功能架构
 - 三、Redis 数据结构与业务场景对应
 - 1. String 类型
 - 2. Hash 类型
 - 3. List 类型
 - 4. Set 类型
 - 5. ZSet 类型 (有序集合)
 - 6. HyperLogLog 类型
 - 四、核心功能详解
 - 4.1 库存防超卖 (Redis 原子操作) ★★ ★
 - 【高并发防超卖架构设计】
 - 【完整业务流程】
 - 【为什么 Redis 能防超卖? 】
 - 【数据一致性保障】
 - 【实际代码位置】
 - 4.2 缓存三大问题解决方案
 - 缓存穿透
 - 缓存击穿
 - 缓存雪崩
 - 4.3 延迟队列 (订单超时处理)
 - 4.4 热搜排行榜 (ZSet)
 - 4.5 PV/UV 统计
 - 五、API 接口说明
 - Redis 功能演示接口
 - 六、文件清单
 - 新增文件
 - 修改文件

TomatoMall Redis 功能说明文档

一、概述

本项目集成了 Redis 缓存系统，实现了多种 Redis 数据结构的应用场景，体现对 Redis 的深入理解。

二、Redis 功能架构

Redis 功能模块		
基础功能(4个)	进阶功能(3个)	高级功能(3个)
- 用户会话 (String) - 商品详情缓存 (Hash) - 商品收藏 (Set) - 首页广告缓存 (List)	- 库存防超卖 (String原子) - 购物车 (Hash) - 商品热搜榜 (ZSet)	- 缓存三大问题解决方案 (穿透/击穿/雪崩) - 超时订单延迟队列 (ZSet) - PV/UV 统计 (String + HyperLogLog)

三、Redis 数据结构与业务场景对应

1. String 类型

应用场景	Key 格式	说明
库存缓存	stock:product:{productId}	使用 INCR/DECR 原子操作防超卖
PV 统计	stats:pv:{pageKey}	使用 INCR 原子递增
分布式锁	lock:product:{productId}	使用 SETNX 实现互斥

核心命令: SET, GET, INCR, DECR, SETNX, EXPIRE

2. Hash 类型

应用场景	Key 格式	说明
商品详情缓存	<code>product:detail:{productId}</code>	存储商品多个属性
购物车	<code>cart:user:{userId}</code>	Field=商品ID, Value=数量

核心命令: `HSET`, `HGET`, `HGETALL`, `HINCRBY`, `HDEL`

3. List 类型

应用场景	Key 格式	说明
广告列表缓存	<code>ad:homepage:list</code>	有序的广告列表

核心命令: `LPUSH`, `RPUSH`, `LRANGE`, `LPOP`, `RPOP`

4. Set 类型

应用场景	Key 格式	说明
用户收藏	<code>favorite:user:{userId}</code>	自动去重, O(1) 判断

核心命令: `SADD`, `SREM`, `SMEMBERS`, `SISMEMBER`, `SCARD`

5. ZSet 类型 (有序集合)

应用场景	Key 格式	说明
热搜关键词	<code>search:hot:keywords</code>	Score=搜索次数
热门商品	<code>product:hot</code>	Score=浏览次数
延迟队列	<code>delay:queue:order:timeout</code>	Score=执行时间戳

核心命令: `ZADD`, `ZINCRBY`, `ZREVRANGE`, `ZRANGEBYSCORE`, `ZREM`

6. HyperLogLog 类型

应用场景	Key 格式	说明
UV 统计	<code>stats:uv:{pageKey}</code>	去重计数，固定 12KB

核心命令: `PFADD`, `PFCOUNT`, `PFMERGE`

四、核心功能详解

4.1 库存防超卖 (Redis 原子操作) ★ ★ ★

【高并发防超卖架构设计】

传统方案的问题:

```
// ❌ MySQL 方案（存在并发问题）
Stockpile stock = stockRepository.findById(productId); // 1. 读取
if (stock.getAmount() >= quantity) { // 2. 判断
    stock.setAmount(stock.getAmount() - quantity); // 3. 更新
    stockRepository.save(stock); // 4. 写入
}
// 问题：步骤1-4之间，其他线程可能修改库存，导致超卖
```

本项目实现的 Redis 方案:

```
// ✅ Redis 原子操作（彻底解决并发问题）
Long newStock = stockCacheService.decrStock(productId, quantity);
// 底层执行: DECRBY stock:product:51 1
// Redis 单线程模型保证命令串行执行，不会出现竞态条件

if (newStock < 0) {
    // 库存不足，原子回滚
    stockCacheService.incrStock(productId, quantity);
    throw new RuntimeException("库存不足");
}
// 订单创建...
```

【完整业务流程】

高并发下单流程设计

1. 用户下单
↓
2. 【Redis DECR】原子扣减库存 ← 防超卖核心
 - ├ 成功($\text{newStock} \geq 0$) → 继续流程
 - └ 失败($\text{newStock} < 0$) → INCR 回滚, 返回库存不足
↓
3. 【MySQL】创建订单记录 (订单状态=待支付)
↓
4. 【延迟队列】30分钟后检查订单状态
 - ├ 已支付 → 同步扣减 MySQL 库存
 - └ 未支付 → Redis INCR 恢复库存, 取消订单

【为什么 Redis 能防超卖？】

对比项	MySQL 方案	Redis 方案
并发性能	悲观锁: 500 QPS 乐观锁: 1000 QPS	DECR 原子: 100,000 QPS
实现方式	SELECT FOR UPDATE (行锁) 或 Version 字段	单线程命令串行执行
超卖风险	乐观锁有重试开销 悲观锁易死锁	完全无并发问题
数据库压力	高并发下锁等待	Redis 内存操作, 毫秒级

对比 MySQL:

```
// MySQL 中的"读-改-写" (即使在事务中也非原子)
SELECT amount FROM stockpile WHERE id = 1; // 其他线程可以在这里插入
UPDATE stockpile SET amount = amount - 1 WHERE id = 1;
// 需要加锁才能保证原子性: SELECT ... FOR UPDATE
```

【数据一致性保障】

1. Redis 为主 (实时库存, 防超卖)
2. MySQL 为从 (持久化备份, 支付后同步)
3. 最终一致性:
 - 下单时: 只扣 Redis
 - 支付成功: 同步扣 MySQL
 - 订单取消: Redis INCR 恢复

【实际代码位置】

- 实现: `OrderServiceImpl.createOrder()` (第 40-110 行)
- 核心服务: `StockCacheServiceImpl.decrStock()`
- 回滚逻辑: `rollbackRedisStock()` 私有方法

核心命令: `DECRBY`, `INCRBY`, `GET`, `SET`

4.2 缓存三大问题解决方案

缓存穿透

问题: 查询不存在的数据, 每次都打到数据库

解决: 缓存空值

```
if (product == null) {  
    // 缓存空值标记, 5分钟过期  
    redis.hSet(key, "_null", "NULL_OBJECT", 5, TimeUnit.MINUTES);  
}
```

缓存击穿

问题: 热点数据过期瞬间, 大量请求打到数据库

解决: 互斥锁重建缓存

```
String lockKey = "lock:product:" + productId;  
boolean locked = redis.setIfAbsent(lockKey, "1", 10, TimeUnit.SECONDS);  
if (locked) {  
    try {  
        // 从数据库加载并缓存  
        loadAndCache(productId);  
    } finally {  
        redis.delete(lockKey);  
    }  
}
```

缓存雪崩

问题: 大量缓存同时过期, 请求全部打到数据库

解决: 随机过期时间

```
// 基础时间 + 随机值
long ttl = 30 + new Random().nextInt(10); // 30-40分钟
redis.expire(key, ttl, TimeUnit.MINUTES);
```

4.3 延迟队列 (订单超时处理)

实现原理:

1. 下单时: `ZADD delay:queue:order:timeout <当前时间+30分钟> "order:123"`
2. 定时轮询: `ZRANGEBYSCORE ... 0 <当前时间>` 获取到期任务
3. 处理后删除: `ZREM delay:queue:order:timeout "order:123"`

```
// 添加延迟任务
public void addOrderTimeoutTask(Integer orderId, int timeoutMinutes) {
    long executeTime = System.currentTimeMillis() + timeoutMinutes * 60 * 1000L;
    redis.zAdd("delay:queue:order:timeout", "order:" + orderId, executeTime);
}

// 获取到期任务
public List<String> getExpiredTasks() {
    return redis.zRangeByScore(key, 0, System.currentTimeMillis());
}
```

4.4 热搜排行榜 (ZSet)

实现原理:

```
// 记录搜索 (原子递增分数)
redis.zIncrScore("search:hot:keywords", keyword, 1);

// 获取 Top 10
Set<Object> top10 = redis.zReverseRange("search:hot:keywords", 0, 9);
```

ZSet 优势:

- 自动按分数排序
- ZINCRBY 原子操作
- $O(\log N)$ 插入/查询

4.5 PV/UV 统计

PV (页面浏览量): String + INCR

```
redis.incr("stats:pv:homepage", 1);
```

UV (独立访客): HyperLogLog

```
// 添加用户（自动去重）
redis.pfAdd("stats:uv:homepage", userId);

// 获取 UV（估算值，误差 0.81%）
Long uv = redis.pfCount("stats:uv:homepage");
```

HyperLogLog 优势:

- 固定 12KB 内存
- 支持亿级去重
- 误差仅 0.81%

五、API 接口说明

Redis 功能演示接口

接口	方法	说明
/api/redis/product/{id}	GET	获取商品（带缓存）
/api/redis/hot/search	POST	记录搜索热度
/api/redis/hot/keywords	GET	获取热搜排行榜
/api/redis/stats/visit	POST	记录页面访问
/api/redis/stats/{pageKey}	GET	获取页面统计
/api/redis/stock/{productId}	GET	获取库存
/api/redis/stock/{productId}/decr	POST	扣减库存
/api/redis/status	GET	获取缓存状态

六、文件清单

新增文件

文件路径	说明
configure/RedisConfig.java	Redis 配置类
configure/RedisCacheWarmer.java	缓存预热
configure/OrderTimeoutTask.java	订单超时定时任务
service/RedisService.java	Redis 基础服务接口
service/serviceImpl/RedisServiceImpl.java	Redis 基础服务实现
service/ProductCacheService.java	商品缓存服务接口
service/serviceImpl/ProductCacheServiceImpl.java	商品缓存服务实现
service/AdvertisementCacheService.java	广告缓存服务接口
service/serviceImpl/AdvertisementCacheServiceImpl.java	广告缓存服务实现
service/FavoriteCacheService.java	收藏缓存服务接口
service/serviceImpl/FavoriteCacheServiceImpl.java	收藏缓存服务实现
service/StockCacheService.java	库存缓存服务接口
service/serviceImpl/StockCacheServiceImpl.java	库存缓存服务实现
service/CartCacheService.java	购物车缓存服务接口

文件路径	说明
service/serviceImpl/CartCacheServiceImpl.java	购物车缓存服务实现
service/HotRankService.java	热搜榜服务接口
service/serviceImpl/HotRankServiceImpl.java	热搜榜服务实现
service/DelayQueueService.java	延迟队列服务接口
service/serviceImpl/DelayQueueServiceImpl.java	延迟队列服务实现
service/StatisticsService.java	统计服务接口
service/serviceImpl/StatisticsServiceImpl.java	统计服务实现
controller/RedisController.java	Redis 演示 Controller

修改文件

文件路径	修改说明
pom.xml	添加 Redis 依赖
application.yml	添加 Redis 配置
controller/ProductController.java	集成商品缓存、热度统计、PV/UV
controller/OrderController.java	集成延迟队列
controller/CartItemController.java	集成延迟队列
controller/AdvertisementController.java	集成广告缓存